Relatório 4 — Encoder e Controle PID

Projeto de Sistemas Robóticos

Artur Fonseca - 2022035520
Douglas Filho - 2020054331
Gabriel Fadoul - 2021031645
Ian Godoi - 2022093024
Mateus Cabral - 2024088680

Universidade Federal de Minas Gerais — 2026

1 Introdução

O controle de velocidade de motores é importante em sistemas robóticos, principalmente quando precisamos que uma roda gire em uma velocidade determinada. Apenas enviar um comando para o motor não garante que ele vai atingir a velocidade desejada, pois o comportamento pode variar dependendo das características do motor, do atrito e das condições de funcionamento.

Neste trabalho, utilizamos um servo motor MG90S de rotação contínua, um encoder e um Arduino Mega para medir e controlar a velocidade de uma roda. O servo é responsável por movimentar a roda por meio de um conjunto de engrenagens, enquanto o encoder permite acompanhar o movimento realizado.

Durante a montagem, utilizamos uma engrenagem de oito dentes no encoder, enquanto os outros grupos utilizaram uma de dez dentes. Essa diferença de 20% na quantidade de dentes alterou a relação de transmissão entre a roda e o encoder, influenciando a quantidade de ticks registrados a cada volta. Essa alteração pode explicar parte da discrepância observada em nossos resultados em comparação com os demais grupos. Por isso, realizamos a calibração do encoder considerando as características da nossa própria montagem.

O encoder funciona contando pulsos gerados durante a rotação. A partir dessa contagem, podemos calcular a distância percorrida pela roda e sua velocidade. Para isso, primeiro precisamos descobrir quantos ticks são registrados em uma volta completa e medir o diâmetro da roda. Com essas informações, conseguimos converter a contagem do encoder em uma distância expressa em milímetros.

Também realizamos testes com diferentes comandos enviados ao servo, para observar como a velocidade varia de acordo com a largura do pulso aplicado. Em seguida, comparamos dois filtros digitais, uma média móvel e um filtro passa-baixa exponencial, para reduzir as oscilações presentes nas medições de velocidade.

O objetivo do trabalho é utilizar essas medições para implementar um controlador PID, capaz de ajustar automaticamente o comando do servo de acordo com a diferença entre a velocidade desejada e a velocidade medida. Dessa forma, buscamos fazer com que a roda acompanhe diferentes velocidades de referência, incluindo mudanças de velocidade e inversões no sentido de rotação.

2 Montagem do Sistema

A montagem do sistema foi feita durante as aulas presenciais no laboratório do ICEX. Durante elas, encontramos diversos problemas: o primeiro deles foi alguns dos parafusos do sistema de rodas, que não apertavam de maneira simples. Outro problema muito sério que tivemos foi a soldagem dos jumpers nas pernas do encoder, tivemos que refazer a solda cerca de 3 vezes, por falhas na maneira em que forma em que fizemos ou por ficar muito frágil ao ponto de uma aula para ou-

tra todos os fios soltarem e termos que refazer do zero. Para ambos esses problemas, o professor pôde nos ajudar em sala de aula mesmo, para evitar que nos atrasássemos no desenvolvimento das outras etapas.

Quanto ao software, em algumas etapas do desenvolvimento do código, tivemos um problema sério de comunicação com o Arduino em que o código não era escrito no mesmo(ocorria um erro de timeout), ao ponto de o notebook de um dos membros não poder ser utilizado no desenvolvimento do mesmo, o que também nos atrasou. O professor também tentou nos ajudar, mas o erro era inconsistente, e só pudemos solucioná-lo ao trocar o notebook que utilizávamos, onde o erro até ocorria, mas de maneira mais esporádica. Seguem as imagens do desenvolvimento do sistema:

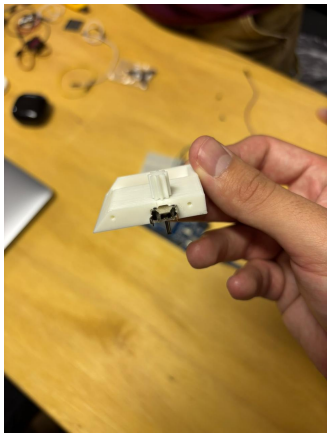


Figura 1: Foto dos conectores do encoder acomplado á peça auxiliar impressa

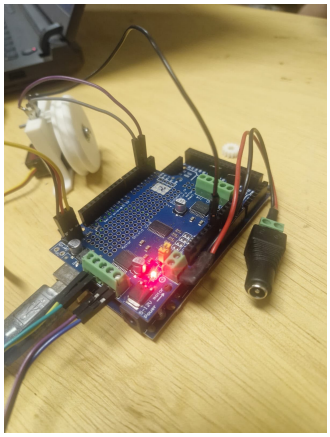


Figura 2: Foto do sistema completo

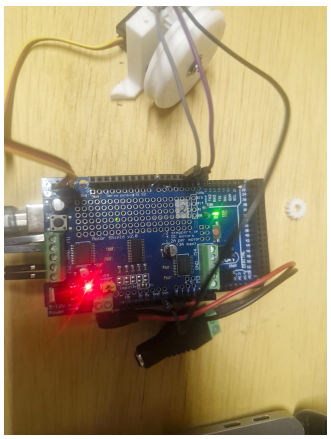


Figura 3: Disposição das portas utilizadas no Arduino

3 Calibração do Encoder e Cálculo da Velocidade

Antes de medir a velocidade da roda, foi necessário descobrir quantos ticks o encoder registrava a cada volta completa. Essa calibração é importante porque o encoder não fornece diretamente a distância percorrida ou a velocidade. Ele apenas registra as mudanças de posição durante o movimento.

Para realizar essa medição, utilizamos a biblioteca `Encoder.h` no Arduino Mega. Os canais A e B do encoder foram conectados aos pinos digitais 2 e 3, permitindo acompanhar a contagem e identificar o sentido de rotação.

3.1 Medição dos ticks por volta

Inicialmente, zeramos o contador do encoder e giramos a roda manualmente uma volta completa. O procedimento foi realizado nos sentidos horário e anti-horário.

No sentido horário, foram registrados 231 ticks. Já no sentido anti-horário, foram registrados -233 ticks. O sinal negativo indica apenas que a roda girou no sentido contrário ao adotado como positivo.

Os resultados estão apresentados na Tabela 1.

Tabela 1 – Contagem do encoder

Sentido	Ticks
Horário	231
Anti-horário	-233
Média dos módulos	232

A diferença entre as duas medições foi de apenas dois ticks. Durante o movimento manual, percebemos que a roda podia retornar um pouco ao soltar ou movimentar as engrenagens, fazendo a contagem aumentar e diminuir em alguns momentos.

Como os valores encontrados foram próximos, utilizamos a média dos módulos:

$$N = \frac{|231| + |-233|}{2} \tag{1}$$

$$\boxed{N = 232 \text{ ticks/volta}} \tag{2}$$

Esse valor foi utilizado nos cálculos seguintes.

3.2 Conversão de ticks em distância

Após determinar a quantidade de ticks por volta, medimos o diâmetro externo da roda, obtendo aproximadamente 4,5 cm, ou 45 mm.

Para descobrir a distância correspondente a uma volta completa, calculamos a circunferência da roda:

$$C = \pi D \tag{3}$$

Substituindo o diâmetro medido:

$$C = \pi \times 45 \tag{4}$$

$$C \approx 141,37 \text{ mm} \tag{5}$$

Portanto, considerando que a roda role sem escorregar, uma volta completa corresponde a aproximadamente 141,37 mm de deslocamento.

Sabendo que uma volta produz, em média, 232 ticks, calculamos a constante de conversão k:

$$k = \frac{\pi D}{N} \tag{6}$$

$$k = \frac{\pi \times 45}{232} \tag{7}$$

$$\boxed{k \approx 0,6094 \text{ mm/tick}} \tag{8}$$

Isso significa que cada tick registrado pelo encoder corresponde a aproximadamente 0,6094 mm de deslocamento da roda.

Com essa constante, podemos calcular a distância a partir da contagem acumulada:

$$s = k \times \text{ticks} \tag{9}$$

Por exemplo, se o encoder registrar 100 ticks:

$$s = 0,6094 \times 100 \tag{10}$$

$$s = 60,94 \text{ mm} \tag{11}$$

3.3 Cálculo da velocidade

Depois de calcular a distância por tick, implementamos a medição de velocidade no Arduino.

O programa realiza leituras do encoder em intervalos de aproximadamente 50 milissegundos. A cada leitura, calculamos quantos ticks foram registrados desde a leitura anterior.

A velocidade é obtida dividindo a distância percorrida pelo intervalo de tempo:

$$v = \frac{\Delta s}{\Delta t} \tag{12}$$

Como a distância depende da variação dos ticks, também podemos escrever:

$$v = \frac{\Delta \text{ticks} \times k}{\Delta t} \tag{13}$$

No código, o intervalo real é calculado utilizando a função `micros()`, evitando considerar que todas as leituras acontecem exatamente no mesmo instante.

Por exemplo, se forem registrados 10 ticks em 50 ms, temos:

$$\Delta s = 10 \times 0,6094 \tag{14}$$

$$\Delta s = 6,094 \text{ mm} \tag{15}$$

Convertendo 50 ms para segundos:

$$\Delta t = 0,05 \text{ s} \tag{16}$$

Assim:

$$v = \frac{6,094}{0,05} \tag{17}$$

$$\boxed{v = 121,88 \text{ mm/s}} \tag{18}$$

3.4 Resolução da medição de velocidade

Como o encoder registra apenas ticks inteiros, a velocidade calculada não varia de maneira contínua. Ela assume valores separados por um determinado intervalo, dependendo do tempo entre leituras.

Esse intervalo, chamado aqui de degrau de velocidade, pode ser calculado por:

$$\Delta v = \frac{k}{\Delta t} \tag{19}$$

Utilizando a constante encontrada e o intervalo de 50 ms:

$$\Delta v = \frac{0,6094}{0,05} \tag{20}$$

$$\boxed{\Delta v \approx 12,19 \text{ mm/s}} \tag{21}$$

Isso significa que, em uma janela de 50 ms, um tick adicional representa aproximadamente 12,19 mm/s na velocidade calculada.

Na prática, mesmo que a roda esteja girando com velocidade aproximadamente constante, o número de ticks registrado em cada intervalo pode variar. Por isso, a velocidade medida também apresenta oscilações.

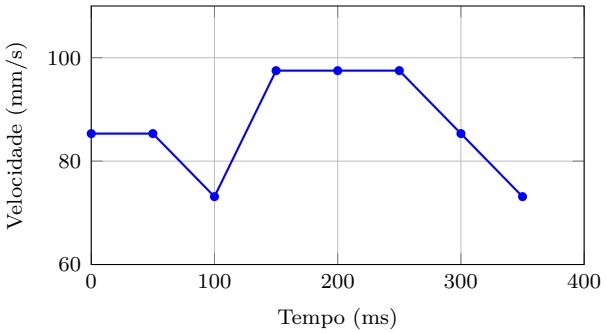


Figura 1 – Oscilações da velocidade calculada a partir do encoder, com pulso de 1600 μs.

A Figura 1 apresenta um trecho das medições realizadas com o servo recebendo um pulso constante de 1600 μs.

Mesmo mantendo o comando do servo, a velocidade calculada apresentou pequenas variações entre as leituras. Isso acontece, em parte, devido à resolução do encoder e ao intervalo de amostragem utilizado.

Essas oscilações motivaram a comparação de filtros digitais, apresentada na Seção 5, com o objetivo de obter uma medida de velocidade mais estável para o controlador PID.

3.5 Resumo da calibração

A Tabela 2 apresenta os principais valores utilizados no cálculo da distância e da velocidade.

Tabela 2 – Parâmetros de calibração

Parâmetro	Valor
Diâmetro da roda	45 mm
Ticks por volta	232
Circunferência	141,37 mm
Distância por tick	0,6094 mm
Intervalo de leitura	50 ms
Degrau de velocidade	12,19 mm/s

4 Caracterização do Servo em Malha Aberta

Após realizar a calibração do encoder, começamos os testes para entender como o servo motor MG90S se comportava com diferentes comandos de velocidade.

Nessa etapa, trabalhamos em malha aberta, ou seja, enviamos um pulso para o servo e medimos a velocidade resultante, sem utilizar um controlador para corrigir possíveis diferenças.

O objetivo foi observar como a velocidade da roda variava de acordo com a largura do pulso enviado pelo Arduino.

4.1 Determinação do pulso neutro

Primeiramente, precisamos encontrar o pulso que mantém o servo parado.

Para isso, utilizamos a função `writeMicroseconds()` da biblioteca `Servo.h`, que permite controlar diretamente a largura do pulso enviado ao motor.

Começamos aplicando um pulso de 1500 μs. Nesse valor, observamos que a roda permanecia parada. Posteriormente, confirmamos esse comportamento por meio do encoder, que registrou velocidade igual a zero durante dez leituras consecutivas.

Também realizamos testes com valores próximos ao neutro. Ao aumentar o pulso, observamos o início de movimento lento em aproximadamente 1550 μs. No sentido contrário, o movimento começou próximo de 1460 μs.

Esses resultados indicam a existência de uma banda morta ao redor do pulso neutro. Nessa região, pequenas alterações no comando não produzem uma rotação contínua da roda.

Os limites encontrados são aproximados, pois não realizamos uma medição detalhada de todos os pulsos intermediários.

Assim, adotamos como referência para os próximos experimentos:

$$\boxed{u_{\text{neutro}} = 1500\ \mu s} \tag{22}$$

4.2 Medição da velocidade para diferentes pulsos

Depois de identificar o pulso neutro, realizamos testes com diferentes larguras de pulso para observar a velocidade da roda.

Foram aplicados comandos de 1000 a 2000 μs, com intervalos de 100 μs. Em cada teste, o servo foi mantido girando até atingir uma velocidade aproximadamente estável.

A velocidade foi calculada a partir da contagem dos ticks do encoder, utilizando a constante de conversão obtida na Seção 3.

Para cada pulso, registramos um conjunto de leituras e calculamos a média das velocidades medidas. Esse procedimento foi utilizado porque a velocidade calculada apresentava pequenas oscilações mesmo quando o comando do servo permanecia constante.

Os resultados estão apresentados na Tabela 3.

Tabela 3 – Velocidade em malha aberta

Pulso (μs)	Velocidade (mm/s)
1000	$-271,78$
1100	$-228,85$
1200	$-186,88$
1300	$-143,55$
1400	$-93,84$
1500	$0,00$
1600	$88,02$
1700	$130,01$
1800	$168,18$
1900	$211,25$
2000	$255,94$

Os valores negativos representam a rotação em um dos sentidos, enquanto os positivos representam a rotação no sentido contrário.

4.3 Curva de velocidade do servo

A partir dos dados obtidos, construímos a curva que relaciona a largura do pulso aplicado com a velocidade média da roda.

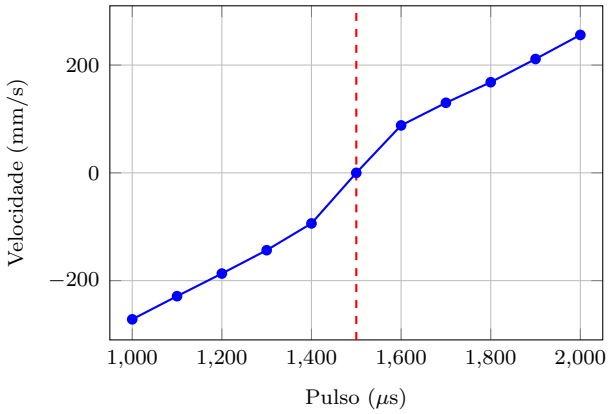


Figura 2 – Relação entre a largura do pulso e a velocidade média da roda em malha aberta. A linha tracejada indica o pulso neutro.

Observando a Figura 2, percebemos que a velocidade aumenta conforme o pulso se afasta do valor neutro.

Para pulsos menores que 1500 μs, a velocidade apresenta valores negativos. Já para pulsos maiores que 1500 μs, a velocidade apresenta valores positivos.

Também podemos perceber que a relação entre o pulso e a velocidade não é perfeitamente linear, principalmente na região próxima ao neutro.

4.4 Banda morta e assimetria

Durante os testes próximos ao pulso neutro, percebemos que o servo não começava a girar imediatamente após pequenas alterações no comando.

O movimento contínuo foi observado próximo de 1460 μs em um sentido e de 1550 μs no sentido contrário.

Isso indica que existe uma faixa de comandos em que o servo permanece parado ou apresenta movimento insuficiente para manter a rotação.

Essa característica é importante para o controle de velocidade, pois o controlador pode tentar corrigir um erro pequeno enviando um pulso que ainda esteja dentro da banda morta.

Outra característica observada foi a diferença entre as velocidades nos dois sentidos de rotação.

Por exemplo, ao aplicar 1400 μs, obtivemos velocidade média de aproximadamente $-93,84$

mm/s. Já com 1600 μs, a velocidade média foi de 88, 02 mm/s.

Apesar de ambos os comandos estarem a 100 μs do neutro, as velocidades não foram exatamente iguais em módulo.

Essa diferença também apareceu em outros valores, indicando uma assimetria na resposta do conjunto formado pelo servo e pelas engrenagens.

4.5 Saturação do servo

Também procuramos observar se a velocidade atingia um limite, mesmo aumentando a largura do pulso.

Dentro da faixa de 1000 a 2000 μs, não identificamos uma região em que a velocidade permanecesse praticamente constante com o aumento do comando.

No sentido positivo, por exemplo, a velocidade média passou de aproximadamente 211,25 mm/s em 1900 μs para 255,94 mm/s em 2000 μs.

Portanto, não foi possível identificar claramente a saturação da velocidade dentro da faixa utilizada no experimento.

Esses resultados mostram que o servo apresenta uma relação bem definida entre o comando aplicado e a velocidade obtida, mas também possui características que precisam ser consideradas no controle, como a banda morta e a diferença de comportamento entre os sentidos.

5 Comparação dos Filtros de Velocidade

Na etapa anterior, observamos que a velocidade calculada pelo encoder apresentava oscilações, mesmo quando o servo recebia um pulso constante. Isso acontece porque a velocidade é calculada a partir da quantidade de ticks registrados em um intervalo de tempo, e essa quantidade nem sempre é igual em todas as leituras.

Como o controlador PID utiliza a velocidade medida para calcular o erro em relação à velocidade desejada, essas oscilações podem prejudicar o controle. Por exemplo, o controlador pode interpretar uma variação causada pela contagem dos ticks como uma mudança real na velocidade da roda.

Para reduzir esse problema, testamos dois filtros digitais: a média móvel simples e o filtro passa-baixa exponencial. A ideia foi comparar a suavização produzida por cada filtro e observar quanto tempo eles demoravam para acompanhar uma mudança de velocidade.

5.1 Média móvel

O primeiro filtro utilizado foi a média móvel simples. Seu funcionamento consiste em calcular a média das últimas medições de velocidade.

Em vez de utilizar somente a leitura mais recente, o filtro considera um conjunto de valores anteriores. Dessa forma, uma medição muito alta ou muito baixa tem seu efeito reduzido pela presença das outras amostras.

A média móvel pode ser representada por:

$$v_{MM}[n] = \frac{1}{M} \sum_{i=0}^{M-1} v[n-i] \tag{23}$$

Nessa equação, $v[n]$ representa a velocidade bruta na amostra atual, e M representa a quantidade de amostras utilizadas para calcular a média.

No nosso experimento, utilizamos uma janela de cinco amostras:

$$\boxed{M = 5} \tag{24}$$

Como o Arduino realiza uma leitura aproximadamente a cada 50 ms, essa janela corresponde a cinco medições consecutivas, abrangendo 250 ms de amostragem.

Por exemplo, se as últimas cinco velocidades medidas fossem 121,88, 134,04, 121,87, 146,26 e 134,05 mm/s, a média móvel seria:

$$v_{MM} = \frac{121,88 + 134,04 + 121,87 + 146,26 + 134,05}{5} \tag{25}$$

$$v_{MM} = 131,62 \text{ mm/s} \tag{26}$$

Nesse exemplo, percebemos que o resultado fica entre os valores medidos, reduzindo as oscilações das leituras individuais.

A desvantagem é que o filtro utiliza valores anteriores. Quando a velocidade muda rapidamente, a média móvel demora um pouco para acompanhar a nova velocidade.

5.2 Filtro passa-baixa exponencial

O segundo filtro utilizado foi um passa-baixa de primeira ordem, implementado por meio de uma média exponencial.

Diferentemente da média móvel, que utiliza uma quantidade fixa de amostras, esse filtro combina a velocidade atual com o resultado filtrado da leitura anterior.

Sua equação é:

$$v_{PB}[n] = \alpha v[n] + (1-\alpha) v_{PB}[n-1] \qquad (27)$$

O parâmetro α determina quanto peso será dado à medição atual.

Quando α é próximo de 1, o filtro responde rapidamente às mudanças, mas suaviza menos as oscilações. Quando α é menor, o resultado fica mais suave, porém demora mais para acompanhar mudanças na velocidade.

No experimento, utilizamos:

$$\boxed{\alpha = 0,2} \qquad (28)$$

Isso significa que cada nova saída do filtro é formada por 20% da velocidade medida naquele instante e 80% do valor filtrado anteriormente.

Os dois filtros foram implementados no mesmo programa, permitindo que recebessem exatamente as mesmas medições do encoder.

5.3 Procedimento experimental

Para comparar os filtros, realizamos um teste de mudança brusca no comando do servo, também chamado de degrau.

Inicialmente, o servo recebeu um pulso de 1500 μs durante dois segundos, mantendo a roda parada.

Após esse intervalo, o Arduino alterou o comando para 1700 μs, fazendo a roda começar a girar. O novo pulso foi mantido durante cinco segundos.

Durante o experimento, o Arduino registrou os ticks do encoder e calculou a velocidade bruta aproximadamente a cada 50 ms.

A mesma velocidade bruta foi enviada aos dois filtros, e os três valores foram registrados pelo Monitor Serial.

Assim, conseguimos comparar a resposta dos filtros durante a partida e também depois que a velocidade da roda ficou aproximadamente constante.

5.4 Resultados obtidos

A Tabela 4 apresenta algumas das medições registradas logo após a mudança de 1500 para 1700 μs.

Tabela 4 – Resposta dos filtros ao degrau

Tempo (ms)	Bruta (mm/s)	MM (mm/s)	PB (mm/s)
2000	0,00	0,00	0,00
2050	48,74	9,75	9,75
2100	97,49	29,25	27,30
2150	134,04	56,05	48,64
2201	121,88	80,43	63,29
2251	121,86	104,80	75,01
2300	134,04	121,86	86,81
2400	146,26	129,18	104,31
2600	121,87	131,62	118,26
3000	134,05	136,50	129,59

Observando os dados, percebemos que a velocidade bruta aumenta rapidamente depois que o servo recebe o novo pulso.

Os dois filtros acompanham esse aumento de maneira mais gradual, pois utilizam informações de medições anteriores.

No instante de 2300 ms, por exemplo, a velocidade bruta era de 134,04 mm/s. A média móvel indicava 121,86 mm/s, enquanto o passa-baixa apresentava 86,81 mm/s.

Isso mostra que, com os parâmetros escolhidos, o filtro passa-baixa apresentou um atraso maior durante a aceleração.

A Figura 4 apresenta a comparação gráfica das três velocidades durante a mudança de comando e parte do regime de rotação.

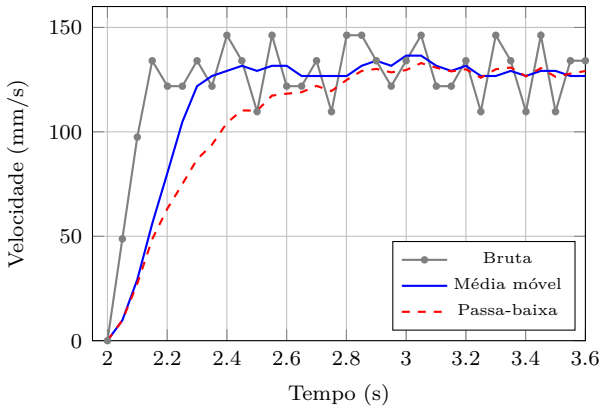


Figura 4 – Comparação entre a velocidade bruta e as saídas dos dois filtros após a aplicação do degrau de 1700 μs.

Na Figura 4, podemos observar que a velocidade bruta apresenta várias oscilações depois da partida.

A média móvel acompanha o aumento da velocidade mais rapidamente, enquanto o passa-baixa demora um pouco mais para se aproximar do valor medido em regime.

Depois que a velocidade se estabiliza, as duas curvas filtradas apresentam oscilações menores que a velocidade bruta.

5.5 Escolha do filtro para o controlador

Após comparar os resultados, decidimos utilizar inicialmente a média móvel com janela de cinco amostras no controlador de velocidade.

Essa escolha foi feita porque, no experimento realizado, a média móvel conseguiu reduzir as oscilações da velocidade e apresentou uma resposta mais rápida ao degrau do que o passa-baixa com $\alpha = 0,2$.

Isso é importante porque o controlador precisa perceber quando a velocidade está abaixo ou acima do valor desejado. Se a medição demorar muito para acompanhar uma mudança, o controlador poderá continuar aplicando correções mesmo depois de a roda já ter acelerado.

Por outro lado, uma velocidade com muitas oscilações também pode provocar alterações desnecessárias no comando do servo.

Por isso, buscamos um equilíbrio entre a suavização das medições e o atraso introduzido pelo filtro.

A escolha da média móvel foi feita com base nesse primeiro experimento. Seu comportamento dentro da malha de controle ainda será avaliado durante a implementação e a sintonia do controlador.

A Tabela 5 resume os parâmetros utilizados.

Tabela 5 – Parâmetros dos filtros

Parâmetro	Valor
Amostragem	50 ms
Média móvel	5 amostras
Passa-baixa (α)	0,2
Filtro escolhido	Média móvel

6 Implementação e Sintonia do Controlador PID

Depois de medir a velocidade da roda e comparar os filtros, começamos a implementar o controle em malha fechada.

Diferentemente dos testes em malha aberta, em que aplicávamos um pulso fixo ao servo, agora o Arduino ajusta esse pulso automaticamente de acordo com a velocidade medida.

Para isso, definimos uma velocidade desejada, chamada de setpoint, e calculamos a diferença entre ela e a velocidade filtrada do encoder.

Essa diferença é o erro do controlador:

$$e(t) = v_{ref}(t) - v_{med}(t) \tag{29}$$

Se a velocidade medida estiver abaixo da desejada, o erro será positivo e o controlador deverá aumentar o comando do servo. Se a velocidade ultrapassar a referência, o erro será negativo e o comando deverá ser reduzido.

O controlador PID utiliza três termos: proporcional, integral e derivativo. Sua equação é:

$$u(t) = K_p e(t) + K_i \int e(t)\,dt - K_d \frac{dv_{med}(t)}{dt} \tag{30}$$

A saída $u(t)$ representa uma correção, em microssegundos, somada ao pulso neutro do servo.

Como encontramos um pulso neutro de 1500 μs, o comando enviado ao motor é:

$$P(t) = 1500 + u(t) \tag{31}$$

Para evitar comandos muito elevados, limitamos a correção a ± 250 μs. Também mantivemos os limites gerais do pulso entre 1000 e 2000 μs.

O intervalo de controle foi de aproximadamente 50 ms, utilizando o tempo real medido entre as leituras. Para a velocidade utilizada no cálculo do erro, adotamos a média móvel de cinco amostras, escolhida na etapa anterior.

6.1 Ajuste do ganho proporcional

Inicialmente, utilizamos apenas o termo proporcional, mantendo os ganhos integral e derivativo iguais a zero:

$$K_i = 0, \qquad K_d = 0 \tag{32}$$

Nesse caso, a correção é calculada por:

$$u(t) = K_p e(t) \tag{33}$$

Definimos uma velocidade desejada de 100 mm/s e realizamos testes com diferentes valores de K_p, mantendo cada experimento durante oito segundos.

Começamos com $K_p = 1$. Nesse teste, a velocidade ficou próxima de 57 mm/s, apresentando um erro elevado em relação à referência. Posteriormente, identificamos um problema na implementação da média móvel utilizada nesse primeiro teste, por isso seu resultado foi mantido apenas como registro inicial.

Após corrigir o filtro, realizamos os testes seguintes. Com $K_p = 2$, a velocidade ficou próxima de 66 mm/s. Aumentando para $K_p = 4$, obtivemos aproximadamente 76 mm/s.

Com $K_p = 5$, a velocidade ficou próxima de 80 mm/s, ainda abaixo dos 100 mm/s desejados.

Também testamos $K_p = 8$. Nesse caso, o controlador passou a apresentar oscilações contínuas. A velocidade ultrapassava a referência, o comando diminuía muito e a roda desacelerava. Em seguida, o controlador aumentava novamente o pulso, repetindo esse comportamento.

A Tabela 6 apresenta um resumo dos testes.

Tabela 6 – Testes do controle proporcional

K_p	Vel. aprox.	Comportamento
1,0	57 mm/s	Erro elevado*
2,0	66 mm/s	Estável
4,0	76 mm/s	Estável
5,0	80 mm/s	Oscilações
8,0	–	Oscilação contínua

*Teste realizado antes da correção do filtro.

Observamos que aumentar o ganho proporcional reduzia o erro, mas também tornava as correções mais intensas.

O ganho $K_p = 8$ apresentou oscilações muito maiores que os anteriores. Por isso, decidimos voltar para $K_p = 5$ e acrescentar o termo integral.

6.2 Adição do termo integral

Mesmo com $K_p = 5$, a velocidade permanecia aproximadamente 20 mm/s abaixo da referência.

Isso acontece porque, no controle proporcional, a correção depende apenas do erro atual. Quando a velocidade aumenta, o erro diminui e o controlador reduz o comando enviado ao motor.

Para tentar corrigir esse comportamento, acrescentamos o termo integral, passando a utilizar um controlador PI:

$$u(t) = K_p e(t) + K_i \int e(t)\, dt \tag{34}$$

O termo integral acumula o erro ao longo do tempo. Assim, enquanto a velocidade permanecer abaixo da referência, ele continua contribuindo para aumentar a correção.

Na implementação, utilizamos a aproximação discreta:

$$I[n] = I[n-1] + e[n]\Delta t \tag{35}$$

O intervalo Δt foi calculado a partir do tempo real entre as leituras.

Também limitamos o valor acumulado da integral para evitar que ele crescesse indefinidamente. Esse cuidado é importante principalmente quando o controlador atinge os limites do pulso.

6.3 Testes com o controlador PI

Mantivemos o ganho proporcional em $K_p = 5$ e começamos utilizando $K_i = 1$.

Nesse teste, a velocidade ficou próxima de 94 mm/s na parte final do experimento. O erro, que anteriormente era de aproximadamente 20 mm/s, caiu para cerca de 6 mm/s.

Depois aumentamos o ganho integral para $K_i = 2$.

Com esse valor, a velocidade ficou ainda mais próxima da referência, oscilando ao redor de 100 mm/s. No trecho final do experimento, observamos valores como 97,50, 99,94 e 102,37 mm/s.

A Tabela 7 resume os resultados.

Tabela 7 – Comparação dos controladores

K_p	K_i	K_d	Vel. aprox.
5,0	0,0	0,0	80 mm/s
5,0	1,0	0,0	94 mm/s
5,0	2,0	0,0	98 mm/s

Os valores apresentados são aproximações obtidas a partir dos registros experimentais. Apesar da melhora, ainda observamos pequenas oscilações na velocidade medida.

6.4 Análise da resposta do controlador PI

No teste com $K_p = 5$ e $K_i = 2$, observamos uma aceleração inicial mais intensa, seguida de oscilações enquanto o controlador ajustava o pulso.

Durante a partida, a velocidade filtrada ultrapassou momentaneamente os 100 mm/s. Em uma das leituras, por exemplo, chegou a 109,69 mm/s.

Depois de alguns segundos, as oscilações diminuíram e a velocidade passou a permanecer mais próxima da referência.

A Figura 5 apresenta um trecho final desse experimento, comparando a velocidade filtrada com a velocidade desejada.

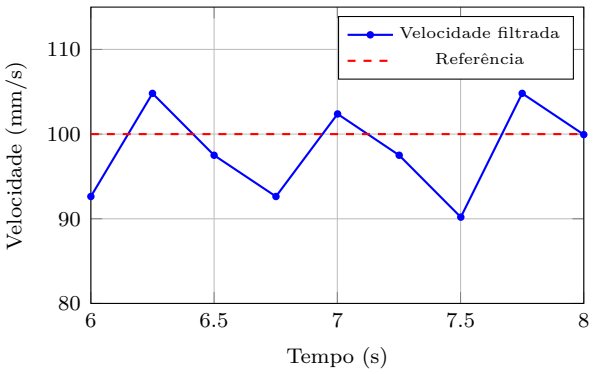


Figura 5 – Trecho final da resposta do controlador PI com $K_p = 5$ e $K_i = 2$.

O gráfico mostra que a velocidade não permanece exatamente em 100 mm/s, mas oscila próxima desse valor.

Esse comportamento é esperado, considerando as variações da velocidade medida pelo encoder e as correções aplicadas pelo controlador.

6.5 Ganhos escolhidos

Com base nos testes realizados, escolhemos inicialmente os seguintes ganhos para os próximos experimentos:

$$\boxed{K_p = 5, \quad K_i = 2, \quad K_d = 0} \tag{36}$$

O termo derivativo não foi utilizado porque o controlador PI já apresentou uma redução considerável do erro, sem oscilações contínuas como as observadas no teste proporcional com $K_p = 8$.

Além disso, como a velocidade é calculada a partir de ticks inteiros, o termo derivativo poderia aumentar a influência das oscilações presentes nas medições.

Os ganhos escolhidos ainda precisam ser avaliados nos ensaios com diferentes velocidades de referência e inversão de sentido, apresentados na próxima seção.

7 Resultados dos Ensaios

7.1 Ensaio de Cinco Patamares

O primeiro ensaio teve como objetivo verificar se o controlador PI conseguia acompanhar diferentes velocidades de referência.

Utilizamos os ganhos $K_p = 5$, $K_i = 2$ e $K_d = 0$, mantendo o filtro de média móvel com cinco amostras.

A roda começou parada e, depois de um segundo, foram aplicadas cinco velocidades crescentes: 50, 80, 100, 130 e 160 mm/s. Cada patamar foi mantido durante aproximadamente três segundos.

Durante o experimento, registramos o tempo, a velocidade bruta, a velocidade filtrada, o erro, o pulso enviado ao servo, a contagem de ticks e a distância acumulada.

Nos dois primeiros patamares, de 50 e 80 mm/s, observamos oscilações consideráveis. Em alguns momentos, a velocidade chegou a zero, mesmo com uma referência positiva. Isso mostrou que o controlador apresentou dificuldade para manter velocidades baixas de maneira contínua.

No patamar de 100 mm/s, a resposta ficou mais próxima da velocidade desejada. Entretanto, ainda ocorreram oscilações e a velocidade permaneceu abaixo da referência durante boa parte do intervalo.

Nos patamares de 130 e 160 mm/s, o controlador também apresentou erro. No último patamar, o pulso chegou frequentemente ao limite de 1750 μs, impedindo que o controlador aumentasse ainda mais o comando.

Na última leitura, a velocidade filtrada foi de 138,94 mm/s, enquanto a referência era de 160

mm/s. O erro registrado foi de 21,06 mm/s.

Esse resultado mostra uma limitação causada pelo valor máximo permitido para a correção do pulso.

Ao final do experimento, o encoder acumulou 2159 ticks, equivalentes a aproximadamente 1315,70 mm de deslocamento da roda, considerando a constante de conversão obtida na calibração.

Como o ensaio foi realizado com a roda suspensa, essa distância representa o deslocamento calculado pela rotação, e não uma distância medida diretamente no chão.

7.2 Ensaio de Inversão de Sentido

O segundo ensaio teve como objetivo avaliar o comportamento do controlador durante uma inversão no sentido de rotação da roda.

Para isso, utilizamos os mesmos ganhos do experimento anterior, com $K_p = 5$, $K_i = 2$ e $K_d = 0$, mantendo o filtro de média móvel com cinco amostras.

Inicialmente, a roda permaneceu parada durante um segundo. Em seguida, aplicamos uma referência de 100 mm/s durante quatro segundos. Após esse intervalo, alteramos a referência para -100 mm/s, mantendo o novo comando por mais quatro segundos.

O sinal negativo representa a rotação no sentido contrário. Dessa forma, o controlador precisava reduzir a velocidade positiva, passar pelo repouso e acelerar novamente no sentido oposto.

A mudança de referência aconteceu aproximadamente em 5 segundos. Nesse instante, a velocidade filtrada ainda era de $97,49$ mm/s, e o controlador aplicou um pulso de 1250 μs, atingindo o limite inferior permitido.

A primeira velocidade bruta negativa foi registrada em $5,151$ segundos, aproximadamente 150 ms depois do comando de inversão.

Já a velocidade filtrada passou a apresentar valores negativos em $5,201$ segundos. Essa diferença ocorre porque o filtro de média móvel ainda considera as medições positivas anteriores à inversão.

Em $5,301$ segundos, a velocidade filtrada chegou a $-99,94$ mm/s, ficando muito próxima da referência desejada. Portanto, foram necessários aproximadamente 300 ms para atingir uma velocidade filtrada próxima do novo setpoint pela primeira vez.

Apesar disso, a velocidade não permaneceu constante. Após a inversão, observamos oscilações, com valores acima e abaixo de -100 mm/s.

Também percebemos que a contagem acumulada do encoder começou a diminuir após a inversão, confirmando que o sentido do movimento estava sendo identificado corretamente.

O teste mostrou que o controlador conseguiu inverter o sentido da roda, mas ainda apresentou oscilações durante a transição e depois dela. A limitação do pulso e o atraso introduzido pelo filtro influenciaram esse comportamento.

7.3 Distância Percorrida

Além da velocidade, também registramos a distância acumulada da roda durante os dois ensaios anteriores.

Para calcular essa distância, utilizamos a contagem de ticks do encoder e a constante de conversão encontrada na calibração, de aproximadamente $0,6094$ mm por tick.

A distância foi calculada pela expressão:

$$s = k \cdot ticks \tag{37}$$

Como o encoder identifica o sentido de rotação, a contagem pode aumentar ou diminuir. Dessa forma, o valor de s representa o deslocamento acumulado com sinal, tomando a posição inicial como referência.

7.3.1 Distância no ensaio de cinco patamares

No primeiro ensaio, a roda recebeu cinco referências positivas de velocidade: 50, 80, 100, 130 e 160 mm/s.

Como o movimento ocorreu no mesmo sentido, a contagem acumulada do encoder aumentou durante o experimento.

Ao final, foram registrados 2159 ticks, correspondentes a:

$$s = 2159 \times 0,6094 \tag{38}$$

$$\boxed{s \approx 1315,70 \text{ mm}} \tag{39}$$

A Figura 6 apresenta a evolução da distância ao longo do tempo.

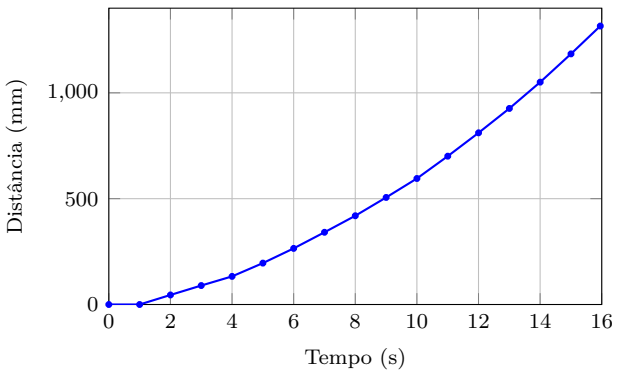


Figura 6 – Distância acumulada no ensaio de cinco patamares.

Observando o gráfico, percebemos que a distância aumenta cada vez mais rapidamente conforme o experimento avança.

Isso pode ser observado pela inclinação da curva. Quando a roda gira mais rápido, ela percorre uma distância maior no mesmo intervalo de tempo, fazendo a curva ficar mais inclinada.

7.3.2 Distância no ensaio de inversão

No segundo ensaio, a roda recebeu inicialmente uma referência de 100 mm/s e depois uma referência de −100 mm/s.

Durante o primeiro movimento, a contagem do encoder aumentou, indicando deslocamento positivo.

Quando o comando foi invertido, a roda começou a girar no sentido contrário e a contagem acumulada passou a diminuir.

A Figura 7 apresenta esse comportamento.

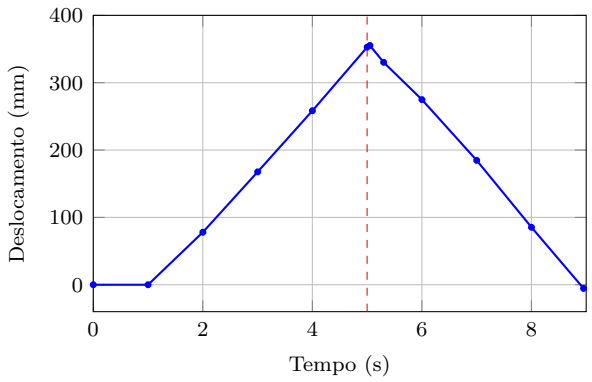


Figura 7 – Deslocamento acumulado durante a inversão de sentido. A linha tracejada indica a mudança de referência.

O gráfico mostra que a distância inicialmente aumenta, até que a roda inverte o sentido de rotação.

A maior posição acumulada registrada foi de aproximadamente 355,28 mm. Depois disso, o deslocamento começou a diminuir.

Ao final do experimento, o encoder registrou −9 ticks, correspondentes a aproximadamente −5,49 mm.

Esse resultado indica que a roda terminou praticamente na mesma posição angular em que começou, mas com uma pequena diferença no sentido negativo.

É importante diferenciar o deslocamento líquido da distância total percorrida. O deslocamento líquido considera a diferença entre a posição final e a inicial, enquanto a distância total considera os movimentos realizados nos dois sentidos.

Considerando a maior posição acumulada e a posição final, estimamos uma distância total de rotação de aproximadamente 716,05 mm, desconsiderando eventuais pequenas inversões intermediárias.

Como os experimentos foram realizados com a roda suspensa, não foi possível comparar diretamente esses valores com uma distância medida no chão.

Portanto, os resultados representam o deslocamento equivalente calculado a partir da rotação da roda, considerando que não exista escorregamento.

8 Discussão

Durante o desenvolvimento do trabalho, tivemos algumas dificuldades, principalmente para entender o comportamento do servo, realizar as medições com o encoder e ajustar o controlador. Apesar de o circuito ser relativamente simples, percebemos que fazer a roda girar era bem diferente de conseguir controlar sua velocidade.

Uma das primeiras dificuldades foi entender o funcionamento do servo de rotação contínua. Diferentemente de um servo convencional, ele não controla diretamente um ângulo, mas utiliza a largura do pulso para determinar o sentido e a velocidade de rotação. No começo, ainda estávamos tentando entender como esses comandos

funcionavam e como fazer o motor parar corretamente. Depois de alguns testes, identificamos que o pulso de 1500 μs correspondia à condição de parada.

Também percebemos que pequenas mudanças no pulso nem sempre produziam movimento. Isso ficou evidente nos testes próximos do valor neutro, mostrando a existência de uma banda morta. Esse comportamento acabou influenciando bastante os resultados, principalmente quando tentamos controlar velocidades menores.

Outra dificuldade foi a calibração do encoder. Como não tínhamos todas as informações sobre a relação entre as engrenagens, precisamos medir diretamente a quantidade de ticks por volta da roda. Fizemos testes nos dois sentidos e encontramos valores próximos de 231 e 233 ticks por volta, adotando 232 como referência. A partir do diâmetro de 45 mm, calculamos a distância correspondente a cada tick.

Essa etapa foi importante porque inicialmente estávamos trabalhando apenas com contagens do encoder. Depois da calibração, conseguimos transformar essas contagens em velocidade e distância, o que facilitou bastante a análise dos experimentos.

Nos testes de velocidade, percebemos que as leituras brutas variavam muito. Como utilizamos um intervalo de aproximadamente 50 ms, pequenas diferenças na quantidade de ticks entre duas amostras provocavam mudanças consideráveis na velocidade calculada. Em alguns momentos, a leitura indicava uma velocidade alta e, logo depois, um valor bem menor, mesmo sem uma mudança tão brusca no movimento observado.

Por causa disso, testamos dois filtros: a média móvel e o filtro passa-baixa exponencial. Os dois conseguiram suavizar as medições, mas também introduziram um atraso na resposta. Com os parâmetros utilizados, a média móvel de cinco amostras respondeu mais rapidamente às mudanças de velocidade, por isso decidimos utilizá-la no controlador.

Mesmo assim, o filtro não eliminou completamente as oscilações. Isso ficou bastante evidente quando começamos os testes de controle proporcional.

Inicialmente, utilizamos apenas o ganho K_p, mantendo os termos integral e derivativo zerados. Começamos com valores menores e fomos aumentando o ganho para observar a resposta. Com $K_p = 2$, a velocidade ficou próxima de 66 mm/s para uma referência de 100 mm/s. Com $K_p = 4$, chegamos a aproximadamente 76 mm/s, e com $K_p = 5$, a velocidade ficou próxima de 80 mm/s.

Durante esses testes, também encontramos um erro na implementação do filtro de média móvel, em que o índice do vetor estava sendo atualizado duas vezes. Precisamos corrigir o código antes de continuar a comparação dos ganhos. Isso mostrou como um erro pequeno na programação pode interferir diretamente nas medições e, consequentemente, no funcionamento do controlador.

Quando aumentamos o ganho proporcional para $K_p = 8$, a resposta piorou bastante. O controlador passou a fazer correções muito fortes, provocando oscilações contínuas. Em alguns momentos, a velocidade ultrapassava a referência e o pulso diminuía tanto que a roda praticamente parava. Logo depois, o controlador tentava compensar aumentando novamente o comando.

Esse teste ajudou a perceber que aumentar o ganho nem sempre melhora o controle. Embora os ganhos maiores reduzissem o erro inicialmente, também podiam deixar o sistema muito instável.

Decidimos então manter $K_p = 5$ e acrescentar o termo integral. Com $K_i = 1$, a velocidade ficou mais próxima dos 100 mm/s desejados, chegando a aproximadamente 94 mm/s no trecho final. Depois, com $K_i = 2$, conseguimos reduzir ainda mais o erro, com a velocidade oscilando próxima de 100 mm/s.

A diferença entre os resultados do controle P e do PI foi uma das partes mais interessantes do experimento. No controle proporcional, a velocidade permanecia abaixo da referência porque a correção diminuía junto com o erro. Com o termo integral, o controlador passou a acumular os erros anteriores e conseguiu manter um comando suficiente para sustentar uma velocidade maior.

Apesar da melhora, o controle PI também não ficou perfeito. Continuamos observando oscilações, principalmente durante as mudanças de velocidade. Por isso, escolhemos os ganhos $K_p = 5$ e $K_i = 2$ como uma configuração para os ensaios finais, e não como valores necessariamente ideais para qualquer situação.

No experimento dos cinco patamares, percebe-

mos que o controlador se comportava de maneira diferente dependendo da velocidade solicitada. Nos patamares de 50 e 80 mm/s, a roda oscilava bastante e chegava a parar em alguns momentos. Acreditamos que isso esteja relacionado à banda morta do servo, às características mecânicas do conjunto e às correções realizadas pelo controlador.

No patamar de 100 mm/s, a resposta foi melhor, embora ainda existisse um erro. Já nos patamares de 130 e 160 mm/s, o controlador não conseguiu alcançar completamente as referências.

No último patamar, observamos que o pulso atingia frequentemente 1750 μs, que era o limite máximo de correção definido no código. Nesse caso, mesmo existindo um erro positivo, o controlador não conseguia aumentar mais o comando. Isso mostrou uma limitação importante do sistema, já que não bastava ajustar os ganhos para alcançar qualquer velocidade desejada.

No experimento de inversão de sentido, tivemos um resultado melhor do que esperávamos. A roda conseguiu passar da rotação positiva para a negativa em pouco tempo. A primeira leitura de velocidade bruta negativa apareceu aproximadamente 150 ms depois da mudança de referência, enquanto a velocidade filtrada chegou próxima de -100 mm/s após cerca de 300 ms.

Também percebemos que o filtro demorava um pouco mais para mostrar a inversão, porque ainda considerava as amostras anteriores. Depois da mudança de sentido, a velocidade apresentou oscilações antes de se aproximar novamente da referência.

Outro resultado interessante foi a contagem acumulada do encoder. No ensaio de inversão, a posição aumentou enquanto a roda girava em um sentido e começou a diminuir quando o movimento foi invertido. Ao final, o deslocamento líquido ficou próximo de zero, mostrando que a roda retornou praticamente à posição angular inicial.

Esse resultado também ajudou a entender a diferença entre distância total percorrida e deslocamento. Embora a posição final tenha ficado próxima da inicial, a roda realizou um movimento considerável nos dois sentidos.

Uma limitação dos nossos experimentos foi que realizamos as medições com a roda suspensa, sem contato com o chão. Dessa forma, conseguimos avaliar o controle de velocidade e a contagem do encoder, mas não verificamos como o conjunto se comportaria com o peso de um robô, atrito com o solo ou possíveis escorregamentos.

Se fôssemos continuar o trabalho, seria interessante testar o controlador com a roda em contato com o chão, investigar melhor a banda morta do servo e experimentar outras combinações de ganhos e filtros. Também seria possível melhorar o tratamento da saturação do controlador, principalmente nos momentos em que o pulso atinge os limites permitidos.

No geral, o trabalho mostrou que controlar a velocidade de uma roda envolve mais do que apenas calcular um erro e enviar um comando ao motor. Precisamos considerar as limitações do hardware, a precisão das medições, o atraso do filtro e a forma como o controlador reage às mudanças de referência.

9 Conclusão

Neste trabalho, conseguimos montar o sistema com o servo de rotação contínua e o encoder, calibrar a contagem de ticks e utilizar essas informações para calcular a velocidade e a distância percorrida pela roda.

Também conseguimos comparar dois filtros e implementar um controlador em malha fechada. Durante os testes, percebemos que o controle proporcional sozinho não foi suficiente para manter a velocidade desejada, enquanto a adição do termo integral reduziu bastante o erro.

Os ganhos escolhidos, $K_p = 5$ e $K_i = 2$, apresentaram resultados razoáveis para uma referência de 100 mm/s, mas o controlador ainda teve dificuldades em velocidades muito baixas e mais altas. Isso ficou evidente no ensaio dos cinco patamares, principalmente quando o pulso atingiu o limite de 1750 μs.

No ensaio de inversão de sentido, conseguimos observar a mudança da velocidade positiva para a negativa e verificar que o encoder registrava corretamente os movimentos nos dois sentidos.

Apesar dos problemas encontrados durante a montagem, a programação e os testes, conseguimos realizar os principais experimentos e entender melhor o funcionamento do controle de velocidade.

A parte mais importante foi perceber, na prática, como os ganhos do controlador, os filtros e as limitações do motor influenciam o resultado.

O sistema ainda pode ser melhorado, principalmente na estabilidade das velocidades menores e no comportamento durante as mudanças de referência, mas os experimentos permitiram aplicar os conceitos de medição, filtragem e controle PID em um sistema real.

Referências

[1] ASTROM, K. J.; MURRAY, R. M. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton: Princeton University Press, 2008. Disponível em: `https://www.cds.caltech.edu/~murray/FBS/Version_2.9d.html`.

[2] BEAUREGARD, B. *Improving the Beginner's PID: Introduction*. 2011. Disponível em: `https://www.datsi.fi.upm.es/docencia/DMC/guia_pid/improving-the-beginners-pid-introduction.htm`.

[3] STOFFREGEN, P. *Encoder Library for Arduino*. GitHub. Disponível em: `https://github.com/PaulStoffregen/Encoder`.

[4] ARDUINO. *Servo Library: writeMicroseconds()*. Documentação da biblioteca Servo. Disponível em: `https://github.com/arduino-libraries/Servo/blob/master/docs/api.md`.